

High Availability and Disaster Recovery

Multi-AZ · Aurora Global DB · Failover · Chaos Engineering · Runbooks · ShopFlow

AWS Tutorial · ShopFlow

20-01

Why High Availability Matters — The Cost of Downtime

ShopFlow processes \$2.3M in orders on a typical day — \$26.60/second. During Black Friday peak, that rises to \$480/second. One minute of downtime at peak costs \$28,800 in lost orders, plus reputational damage and customer churn. High availability is not a technical goal — it is a business requirement.

Availability Level	Annual Downtime	Monthly Downtime	ShopFlow Revenue at Risk (Black Friday peak)
99% ("two nines")	87.6 hours	7.3 hours	This level is unacceptable for an e-commerce platform
99.9% ("three nines")	8.76 hours	43.8 minutes	\$1,382,400/year — still too high
99.95%	4.38 hours	21.9 minutes	\$691,200/year
99.99% ("four nines")	52.6 minutes	4.38 minutes	\$82,944/year — ShopFlow target
99.999% ("five nines")	5.26 minutes	26.3 seconds	\$8,294/year — overkill for cost

■ SLA vs SLO vs SLI

SLI (Service Level Indicator): a metric that measures reliability, e.g., "the percentage of requests returning 2xx in the last 5 minutes." SLO (Service Level Objective): the target you set internally, e.g., "99.99% of requests return 2xx." SLA (Service Level Agreement): a contractual commitment to customers with financial penalties for breaches. ShopFlow targets a 99.99% SLO but only commits to 99.9% SLA to customers — giving engineering headroom.

Example: ShopFlow SLI: measured in CloudWatch as (5xx errors / total requests) per minute. Alert when error rate > 0.01% for 5 consecutive minutes.

20-02

Multi-AZ Architecture — ShopFlow Across Three AZs

An Availability Zone (AZ) is a physically separate data center within an AWS region. Each AZ has independent power, cooling, and networking. AWS regions have 2-6 AZs. ShopFlow uses 3 AZs in us-east-1: us-east-1a, us-east-1b, us-east-1c.



What Fails	Automatic Recovery	Time to Recovery	ShopFlow Behavior
ECS task crashes	ECS service scheduler launches replacement task	30 seconds (if task is in pending state)	ALB sends traffic to other AZs
Entire AZ fails (rare)	ALB detects unhealthy targets in failed AZ via health checks	60 seconds (Stop) + 30 seconds (Rolling restart)	Stop traffic to failed AZ, reroute to other AZs
Aurora writer instance fails	Aurora detects failure, promotes a reader replica	30 seconds (Automatic)	ALB sends traffic to other AZs

20-03

Aurora Global Database — Cross-Region Replication

Aurora Global Database replicates your data from a primary region to up to 5 secondary regions with < 1 second lag. ShopFlow uses it to have a hot standby in eu-west-1 (Ireland) in case us-east-1 (N. Virginia) has a major outage.

Metric	Value	What It Means for ShopFlow
Replication lag	< 1 second (typical: 100-200ms)	EU replica is almost perfectly in sync with US primary. RPO (Recovery Point Objective) is near zero.
Failover time (manual)	< 1 minute	If US region fails, trigger manual failover to EU. EU replica becomes primary.
Failover time (managed)	< 1 minute with RTO SLA	AWS managed planned failover for zero-downtime migrations.
Read performance	EU services can read from local Aurora replica	ShopFlow's future EU expansion: product catalog reads served from EU.

Metric	Value	What It Means for ShopFlow
Cost	Primary region: normal Aurora cost. Secondary region: \$300/month for standby Aurora instance (secondary)	ShopFlow: \$300/month for standby Aurora instance (secondary)

20-04

Multi-Region Failover — Route 53 Health Checks

ShopFlow's production architecture is active-passive: us-east-1 handles 100% of traffic normally. eu-west-1 has a warm standby (ECS services running at minimum capacity, Aurora Global secondary). Route 53 health checks trigger automatic DNS failover.

aws > Route 53 > Health Checks > Create health check

ShopFlow Production Health Check

Primary Region Health Check — us-east-1

Protocol
HTTPS

Domain name
api.shopflow.com

Path
/actuator/health

Port
443

Request interval
10 seconds

Failure threshold
3 consecutive failures = unhealthy

Failover DNS Records

Primary record
api.shopflow.com -> ALB us-east-1 (Failover: PRIMARY, associated with health check above)

Secondary record
api.shopflow.com -> ALB eu-west-1 (Failover: SECONDARY, no health check — always serves if primary fails)

When Route 53 detects 3 consecutive health check failures (30 seconds), DNS TTL expires (60s), and all new connections go to eu-west-1. Total failover time: ~90 seconds. Existing connections to us-east-1 drain normally.

Create health check

20-05

RTO and RPO — ShopFlow's Recovery Targets

Two metrics define disaster recovery: RPO (Recovery Point Objective) is how much data you can afford to lose. RTO (Recovery Time Objective) is how long you can afford to be down. These numbers drive every architectural decision.

Scenario	RPO (Data Loss)	RTO (Downtime)	How Achieved
Single ECS task failure	0 (stateless tasks, no data in task)	< 10 seconds	ECS service scheduler + ALB health checks route around
Single AZ failure	0 (multi-AZ deployment)	< 60 seconds	ALB stops routing to failed AZ. ECS tasks in 2 remaining

Scenario	RPO (Data Loss)	RT0 (Downtime)	How Achieved
Aurora writer instance failure	0 (Aurora synchronous replication to readers)	30 seconds	Aurora automatic failover promotes reader to writer. App retries.
Full region failure (us-east-1 down)	1 second (Aurora Global Replicas)	5 minutes (if annual failover)	Aurora Global promote, Route 53 DNS failover, ECS service failover.
Accidental data deletion (DROP TABLE)	1-5 days (Aurora PITR)	30-60 minutes (restore Aurora PFR)	Aurora Point-in-Time Recovery: restore to any second in time.
Ransomware / S3 data corruption	Up to 30 days (S3 Versioning)	Hours to days (restore S3 replication)	S3 Replication recovers overwritten/deleted objects. Kin

20-06

Chaos Engineering — AWS Fault Injection Simulator

Chaos engineering means intentionally breaking your system in a controlled way to verify your resilience mechanisms actually work. AWS Fault Injection Simulator (FIS) is the AWS tool for this. ShopFlow runs chaos experiments monthly — always in staging first, then production during off-peak.

Experiment	What FIS Does	What ShopFlow Validates	Pass Criteria
ECS task termination	Terminate 50% of product-svc tasks	Aurora can replace tasks	99.9% availability, task 50% down for 20 sec
AZ isolation	Block all network traffic to/from us-east-1a	Aurora can replace tasks	99.9% availability, task 50% down for 20 sec
API latency injection	Add 2-second latency to all calls from ECS to Aurora	Application fails fast (circuit breaker opens)	Latency < 200ms
Redis failure	Terminate primary Redis node	Application falls through to Aurora for data	Latency < 200ms

20-07

Circuit Breaker Pattern — Resilient Service-to-Service Calls

When a downstream service is slow or failing, a naive application keeps sending requests — overloading the failing service and hanging its own threads. The circuit breaker pattern detects failures and "opens the circuit," returning fast errors instead of slow timeouts.

```
// Spring Boot: Resilience4j circuit breaker for product-svc -> Aurora calls // build.gradle:
implementation "io.github.resilience4j:resilience4j-spring-boot3:2.1.0" // application.yml
resilience4j:
  circuitbreaker:
    instances:
      aurora-product-db:
        slidingWindowSize: 20 # evaluate last 20 calls
        failureRateThreshold: 50 # open circuit if 50% fail
        waitDurationInOpenState: 30s # wait 30s before retrying (HALF-OPEN)
        permittedNumberOfCallsInHalfOpenState: 3 # test with 3 calls in HALF-OPEN
        slowCallDurationThreshold: 2s # calls taking > 2s count as failures
        slowCallRateThreshold: 80 # open if 80% are slow // Java service using the circuit breaker
@Service
public class ProductService {
    @CircuitBreaker(name = "aurora-product-db", fallbackMethod = "getProductFromCache")
    public Product getProduct(String productId) {
        return productRepository.findById(productId);
    } // Aurora call .orElseThrow() -> new ProductNotFoundException(productId);
    // Fallback: served from Redis cache when circuit is open
    public Product getProductFromCache(String productId, Exception ex) {
        log.warn("Circuit open for Aurora, serving {} from cache", productId, ex);
        Product cached = redisCache.getProduct(productId);
        if (cached != null) return cached;
        // If also not in cache, return minimal product (degrade gracefully)
        return Product.minimal(productId);
        // name: "Product", price: cached price }
}
```

Circuit State	Condition	Behavior
CLOSED (normal)	Failure rate < 50%. System healthy.	All requests pass through to Aurora normally.
OPEN (failing)	Failure rate exceeded 50% in last 20 calls.	ALL requests immediately return from fallback (cache). No calls to Aurora.
HALF-OPEN (testing)	30-second wait elapsed. Testing if Aurora recovered.	Get test calls sent to Aurora. If they succeed: circuit closes. If they fail: circuit stays open.

20-08

ECS Service Auto-Scaling — Responding to Traffic

ShopFlow's ECS services scale automatically based on CPU utilization and custom metrics. Correct auto-scaling configuration is essential for both availability (scale up fast enough) and cost (scale down when traffic drops).

Parameter	ShopFlow Value	Why
Scale-out metric	CPU utilization > 70% for 2 minutes	2-minute window prevents scaling on brief CPU spikes. 70% leaves 30% headroom.
Scale-in metric	CPU utilization < 30% for 10 minutes	Conservative scale-in threshold: 10-minute window prevents thrashing (scale out then in).
Scale-out cooldown	60 seconds	After adding tasks, wait 60 seconds before evaluating again — new tasks may be added.
Scale-in cooldown	300 seconds (5 minutes)	More conservative than scale-out: keep tasks running longer to absorb peak load.
Minimum tasks	3 (1 per AZ)	Never fewer than 3 tasks: 1 per AZ ensures AZ failure tolerance at all times.
Maximum tasks	50 per service	Prevents runaway scaling from a bug or DDoS. Maximum capacity reviewed quarterly.
Custom metric (checkout-svc)	SQS queue depth > 500 messages	checkout-svc also scales on order queue depth — high queue means orders are piling up.

20-09

ALB Weighted Target Groups — Blue/Green and Canary Deployments

ALB listener rules support weighted routing across target groups, enabling blue/green and canary deployments without Route 53 DNS changes. ShopFlow uses this for zero-downtime deployments of product-svc and checkout-svc.

aws > EC2 > Load Balancers > shopflow-alb > Listeners > HTTPS:443 > Rules

Canary Deployment — product-svc v2.3.1

Weighted Target Group Configuration

Deploying product-svc v2.3.1 as canary: 10% traffic to new version, 90% to current.

Target Group 1: product-svc-v231 (new)

Weight: 10 (10% of matching traffic)

Target Group 2: product-svc-v230 (current)

Weight: 90 (90% of matching traffic)

Monitor CloudWatch: product-svc-v231 error rate for 30 minutes. If 5xx rate < 0.1% -> increase to 50/50 -> then 100/0. If error spike -> set weight to 0/100 (rollback in < 30 seconds, no DNS change needed).

Save changes

Deployment Stage	v2.3.1 Weight	v2.3.0 Weight	Duration	Success Criteria
Canary	10%	90%	30 minutes	Error rate < 0.1%, p99 latency increase < 10%
Partial rollout	50%	50%	30 minutes	Same metrics, both versions consistent
Full rollout	100%	0%	Permanent	Deregister old tasks, delete v2.3.0 target group
Rollback (if needed)	0%	100%	< 30 seconds	Instant: set v2.3.1 weight to 0, no DNS change, no downtime

20-10

Backup Strategy — AWS Backup and Point-in-Time Recovery

ShopFlow uses AWS Backup to centrally manage backups across Aurora, DynamoDB, EFS, and EBS. Every backup policy is defined in code (CDK) and tested quarterly with an actual restore.

Resource	Backup Method	Retention	Restore Time	Last Tested
Aurora PostgreSQL	Automated PITR (continuous) + weekly full backup	35 days (PITR), 60 days (full)	Instant (PITR), 1-2 hours (full)	Quarterly, 2024-09-15
DynamoDB tables	DynamoDB PITR enabled (35-day window)	35 days	Instant	Quarterly, 2024-09-15
S3 (product images, CSVs, etc.)	Versioning enabled on all buckets	1 year	Instant	Quarterly, 2024-09-15

Resource	Backup Method	Retention	Restore Time	Last Tested
EFS (shared config)	AWS Backup daily backup. Cross-region replicated to us-east-1 and us-west-2.	30 days total, 7 days cross-region	< 15 min (restore to new EFS mount point)	Q4 2024

20-11

Incident Response — The ShopFlow P1 Runbook

When the 3am PagerDuty alert fires, this is the 10-step runbook every ShopFlow SRE follows. The goal: restore service fast, capture evidence, fix root cause during business hours.

■ P1 Incident Runbook — ShopFlow Production Outage

1. ('Step 1 — Acknowledge and declare incident (< 2 min)', '# Acknowledge PagerDuty alert\n# Post in #incidents Slack channel:\n"P1 DECLARED: [SERVICE] is down/degraded. I am incident commander. Status page updated to INVESTIGATING."\n# Update status page: status.shopflow.com -> Incident -> shopflow.com is experiencing issues')
2. ('Step 2 — Assess scope (< 5 min)', '# Check CloudWatch dashboards:\n# 1. ShopFlow-Overview: request rate, error rate, p99 latency\n# 2. AWS service health: https://health.aws.amazon.com/health/status\n# 3. ECS services: all tasks healthy? Any recent deployments?\naws ecs describe-services \\\n --cluster shopflow-prod \\\n --services product-svc order-svc checkout-svc \\\n --query "services[*].{service:serviceName,running:runningCount,desired:desiredCount,status:status}"')
3. ('Step 3 — Check recent deployments (< 5 min)', '# Was there a deployment in the last 2 hours?\naws codepipeline list-pipeline-executions \\\n --pipeline-name shopflow-prod-pipeline \\\n --max-results 5\n\n# If yes: rollback immediately (do not wait to investigate)\naws ecs update-service \\\n --cluster shopflow-prod \\\n --service product-svc \\\n --task-definition product-svc:PREVIOUS_VERSION\n\n# ECS rolling update starts immediately; takes < 3 minutes to complete')
4. ('Step 4 — Check ALB and Route 53 (< 5 min)', '# ALB health: any target groups with 0 healthy targets?\naws elbv2 describe-target-health \\\n --target-group-arn ARN_OF_PRODUCT_SVC_TG\n\n# Route 53: is DNS resolving to correct ALB?\n\n# If ALB shows 0 healthy targets: check ECS tasks are running and passing /actuator/health')
5. ('Step 5 — Check Aurora (< 5 min)', '# Aurora cluster status\naws rds describe-db-clusters \\\n --db-cluster-identifier shopflow-aurora-prod \\\n --query "DBClusters[0].{Status:Status,Writer:DBClusterMembers[?IsClusterWriter].DBInstanceIdentifier[0]}"\n\n# If Aurora writer is not available: check for automatic failover in progress\n# Wait up to 60s for Aurora to promote reader. Do NOT force a failover manually during an ongoing auto-failover.')

20-12

Post-Incident Review — Blameless Culture at ShopFlow

Every P1 incident triggers a Post-Incident Review (PIR) within 48 hours. ShopFlow uses blameless post-mortems: the goal is to understand the system failure, not to blame individuals. Engineers are more honest and post-mortems are more useful when people feel psychologically safe.

PIR Section	Content	Example from Real ShopFlow Incident
Summary (5 lines)	What failed, impact, duration, resolution	2024-11-24 14:32 UTC: checkout-svc down 23 minutes. 0 orders processed.
Timeline	Minute-by-minute: first alert, detection, escalation, resolution	14:32 PagerDuty fires. 14:34 On-call declared. 14:38 identified connection pool exhaustion.
Root cause	The technical AND organizational root cause	Technical: analytics query ran FULL TABLE SCAN on orders table (missing index).
Contributing factors	System conditions that made the failure worse	Monitoring: no alert on Aurora connection pool utilization. Process: no test for full table scan.
Action items	Specific, assigned, time-bounded fixes	(1) Add connection pool utilization alarm (P2, @sre-team, done in 3 days).

20-13

AWS Well-Architected Framework — The 6 Pillars

The AWS Well-Architected Framework is AWS's guide to building secure, high-performing, resilient, and efficient infrastructure. It has 6 pillars. ShopFlow's architecture is designed to address all six.

Pillar	Core Question	ShopFlow Implementation
Operational Excellence	Can you run and monitor systems effectively to meet business requirements?	On-call rotation (no on-call time over 12 hours). CloudWatch dashboards. Automated testing.

Pillar	Core Question	ShopFlow Implementation
Security	How do you protect data and systems?	WAF on ALB. Cognito authentication. IAM roles with least privilege (no hardcoded keys).
Reliability	How do you recover from failures?	Multi-AZ ECS, Aurora, ElastiCache. Circuit breakers in application code. Auto-scaling.
Performance Efficiency	How do you use resources efficiently?	ElastiCache Redis for hot data. CloudFront CDN for static assets. OpenSearch for analytics.
Cost Optimization	How do you avoid unnecessary costs?	Compute Savings Plans. Graviton migration. Right-sizing via Compute Optimizer.
Sustainability	How do you minimize environmental impact?	Graviton uses less power than x86. Auto-scaling runs only needed capacity (no always-on).

20-14

Review Q&A

Q	A
What is the difference between RPO and RTO?	RPO (Recovery Point Objective): how much data you can afford to lose (measured in time). RTO (Recovery Time Objective): how long it takes to get back online.
How does ShopFlow achieve RPO < 1 second?	Aurora Global Database for storage-level replication with < 1 second lag to the secondary region.
What is a circuit breaker and why does ShopFlow use it?	A circuit breaker prevents calls to a downstream service. When failures exceed a threshold (50% of requests fail), it stops calls to prevent cascading failures.
How does ALB weighted routing enable zero-downtime deployments?	ALB can split traffic across multiple target groups by percentage (e.g., 10%/90%). ShopFlow deploys new versions to a new target group and gradually shifts traffic.
What does chaos engineering accomplish in ShopFlow?	It verifies system resilience under high traffic. Chaos engineering verifies behavior when components fail.

20-15

Final Production Readiness Checklist — The Complete ShopFlow

This is the comprehensive checklist for taking ShopFlow (or any production e-commerce system) to 99.99% availability. Every chapter of this tutorial contributed one or more items to this list.

Chapter	Topic	Key Checklist Item
Ch01	AWS Foundations	VPC with public/private subnets across 3 AZs. NAT Gateways in each AZ. No resources in default VPC.
Ch02	S3 & CloudFront	S3 Versioning on all buckets. CloudFront in front of all static assets. Cache-Control headers optimized.
Ch03	EC2 & Auto Scaling	No manual EC2 management — use ECS Fargate instead. If EC2 required: ASG with multi-AZ, latest AMI.
Ch04	RDS & Aurora	Aurora Multi-AZ. 35-day PITR. Cross-region snapshot copy. Reserved Instances purchased. Conn. limit increased.
Ch05	ElastiCache	Redis Cluster Mode with 3 nodes across 3 AZs. AUTH token enabled. TLS in transit. Reserved Nodes.
Ch06	DynamoDB	PITR enabled on all tables. Global Tables for active-active multi-region (sessions). On-demand mode.
Ch07	API Gateway	Usage plans and API keys for third-party consumers. Throttling limits set. WAF associated. Custom domain.
Ch08	Cognito	User Pool MFA optional (required for admin users). Password policy enforced. JWT rotation handled.
Ch09	Lambda	All Lambda functions have DLQ configured. Concurrency limits set (prevent runaway). ARM64 (Graviton).
Ch10	SQS/SNS	SQS DLQ on all queues. Message retention: 14 days. Visibility timeout = 6x Lambda max duration.
Ch11	ECS Fargate	Minimum 3 tasks across 3 AZs. Auto-scaling (CPU > 70% scale out, < 30% scale in). Container Insights.
Ch12	OpenSearch	Multi-AZ (3 data nodes, 1 per AZ). Fine-grained access control. Automated snapshots to S3. VPC endpoint.
Ch13	Route 53 & ALB	Alias records (not CNAME) for ALB. Health checks on primary region. Failover routing to secondary region.
Ch14	CloudWatch & X-Ray	Alarms on all critical metrics. X-Ray sampling enabled (5% normal, 100% on errors). Dashboard for all services.
Ch15	CDK	All infrastructure in CDK. CDK Pipelines for self-mutating pipeline. CDK Nag for security checks. No console CLI.
Ch16	CodePipeline	Blue/green ECS deployments. Automated rollback on CloudWatch alarm breach. All deployments via pipeline.
Ch17	WAF & Security	WAF on ALB and CloudFront. AWS Managed Rules enabled. Rate limiting: 2,000 req/5min/IP. WAF logs to S3.
Ch18	Kinesis & Analytics	Kinesis retention: 7 days. Enhanced Fan-Out for multiple consumers. CloudWatch alarms on IterationCount.

Chapter	Topic	Key Checklist Item
Ch19	Cost Optimization	Cost allocation tags on all resources. Monthly budget alert at 80%. Compute Savings Plans covered
Ch20	HA & DR	Aurora Global Database with secondary in eu-west-1. Route 53 health checks for automatic failover

20-16 Disaster Recovery Strategies — Four Tiers

AWS defines four DR tiers in order of increasing cost and decreasing RTO/RPO. ShopFlow's choice reflects the balance between Black Friday revenue risk and infrastructure cost.

DR Strategy	RTO	RPO	Monthly Cost Overhead	ShopFlow Use
Backup & Restore	Hours (restore from backup/last snapshot)	Hours (last snapshot)	Very low (~5-10% extra)	NOT used for production — acceptable only for test/dev
Pilot Light	10-30 minutes (scale up minimal standby env)	Minutes (minimal standby env)	Low (~15-25% extra: replication + minimal standby)	NOT used for production — too slow for Black Friday
Warm Standby	1-5 minutes (scale up second region standby)	< 1 second (constant by replication)	Medium (30-50% extra: full standby by ShopFlow capacity)	Used for production — eu-west-1 runs warm standby
Active-Active (Multi-Region)	Global traffic	0 seconds (continuous replication)	High (~100% extra: duplicate full production)	NOT used for production — ShopFlow Phase 2 goal. Re-

■ Warm Standby vs Active-Passive

ShopFlow's eu-west-1 is a "warm standby": ECS services run with minimum task counts (1 per service instead of 3), Aurora Global secondary is up and receiving replication, Route 53 failover record points to eu-west-1 ALB. Failover requires: (1) Route 53 DNS fails over automatically (~90 seconds). (2) Aurora Global manually promoted (~60 seconds). (3) ECS auto-scaling scales up eu-west-1 tasks based on actual traffic (~3 minutes). Total RTO: ~5 minutes.

Example: Warm standby cost: ~\$3,200/month (minimal ECS tasks + Aurora secondary reader + eu-west-1 ALB + NAT Gateways). Insurance: Black Friday revenue at risk if us-east-1 down = \$28,800/minute.

20-17 Game Day — Quarterly DR Failover Test

A disaster recovery plan that has never been tested is just a hope. ShopFlow runs a quarterly "Game Day" where the team practices the full us-east-1 -> eu-west-1 failover in staging, then a limited production simulation. The test takes 3 hours.

■ Game Day Runbook — Quarterly DR Test

- ('Phase 1 — Staging failover (1 hour)', '# 1. Verify eu-west-1 staging resources are running\naws ecs describe-services --cluster shopflow-staging-eu --region eu-west-1 \\\n --query "services[*].{service:serviceName,running:runningCount}"\n\n# 2. Block all traffic to us-east-1 (Route 53 health check override)\naws route53 change-resource-record-sets \\\n --hosted-zone-id ZONE_ID \\\n --change-batch file://failover-to-eu.json\n\n# 3. Verify traffic flowing to eu-west-1\ncurl https://api-staging.shopflow.com/actuator/health\n\n Expected: {"region":"eu-west-1","status":"UP"}\n\n# 4. Place 10 test orders through eu-west-1 staging environment\n\n# Verify orders appear in eu-west-1 Aurora (now promoted to writer)')
- ('Phase 2 — Aurora Global promote (30 min)', '# Promote Aurora Global secondary to standalone writer\naws rds failover-global-cluster \\\n --global-cluster-identifier shopflow-global \\\n --target-db-cluster-identifier shopflow-aurora-eu\n\n# Wait for promotion to complete (< 60 seconds)\naws rds describe-global-clusters \\\n --global-cluster-identifier shopflow-global \\\n --query "GlobalClusters[0].GlobalClusterMembers[*].{ARN:DBClusterArn,Writer:IsWriter}"\n\n# Verify eu-west-1 Aurora is now writer\n\n# Run read/write queries to confirm')
- ('Phase 3 — Failback (1 hour)', '# After test: restore traffic to us-east-1\naws route53 change-resource-record-sets \\\n --hosted-zone-id ZONE_ID \\\n --change-batch file://failback-to-us.json\n\n# Re-establish Aurora Global: re-add us-east-1 as secondary (30-60 min full resync)\naws rds create-global-cluster \\\n --global-cluster-identifier shopflow-global \\\n --source-db-cluster-identifier shopflow-aurora-us\n\n# Verify both regions in sync before marking Game Day complete\n\n# Write test event to us-east-1, verify it appears in eu-west-1 within 2 seconds')

20-18 Monitoring High Availability — ShopFlow Availability Dashboard

ShopFlow's CloudWatch availability dashboard is the first thing every SRE opens when they start their shift and the last thing they check before going off-call. It shows the health of every resilience layer in one view.

Dashboard Widget	Metric Source	Green / Yellow / Red Threshold	Action if Red
Overall API availability	ALB: HTTPCode_ELB_5XX_Count / RequestErrors	Green: 0.0% Yellow: 0.1% Red: 0.1% (PI status: 0.2%)	Check CloudWatch for spikes in errors. If spike, check logs for errors.
ECS task health per service	ECS: RunningTaskCount per service	Green: running == desired. Yellow: running < desired. Red: running < 1.	Check if task stopped by AWS or by user. If stopped by AWS, check logs for errors.
Aurora replication lag (DBA)	AuroraGlobalDBReplicationLag	Green: < 100ms Yellow: 100-1000ms Red: > 1000ms	Check if replication lag is high. If high, check logs for errors.
ElastiCache cache hit rate (DBA)	ElastiCache: CacheHitRate	Green: > 90%. Yellow: 80-90%. Red: < 80%.	Check if hit rate is low. If low, check logs for errors.
Route 53 health check status	Route53: HealthCheckStatus	Green: healthy. Yellow: unhealthy. Red: unhealthy.	Check if health check is failing. If failing, check logs for errors.
Kinesis iterator age	Kinesis: GetRecords.IteratorAge	Green: < 10s Yellow: 10-60s Red: > 60s	Check if iterator age is high. If high, check logs for errors.

20-19

Series Wrap-Up — What You Have Built with ShopFlow

Across 20 chapters, you have built a complete, production-grade AWS architecture for ShopFlow — an e-commerce platform serving 1 million monthly active users. Here is what every layer does and the chapter that taught it.

Layer	AWS Services	Chapter	ShopFlow Role
Content Delivery	S3 + CloudFront	Ch02	Product images, CSS/JS assets served at < 10ms from edge to 99% of users.
Database	Aurora PostgreSQL 15 (Multi-AZ)	Ch04	Source of truth: orders, products, users. PITR for 35 days. Reserved Instance for cost.
Caching	ElastiCache Redis (3-node cluster)	Ch05	Product catalog, sessions, trending data. 94% cache hit rate reduces DB load.
NoSQL	DynamoDB (Global Tables)	Ch06	Shopping cart, session state, live metrics. Sub-millisecond reads. Auto-scaling.
Container Runtime	ECS Fargate + ALB	Ch11	8 microservices (product, order, checkout, user, search, notification, cart, analytics).
Search	OpenSearch 2.x	Ch12	Full-text product search with facets, typo-tolerance, personalized ranking.
Auth	Cognito User Pools + WAF	Ch17	JWT authentication. Google OAuth. WAF blocks SQL injection, XSS, DDoS.
Async Messaging	SQS + SNS + EventBridge	Ch10	Order processing pipeline, email/SMS notifications, event-driven microservices.
Serverless	Lambda + API Gateway	Ch09	Trending engine, fraud detection, image processing, webhook handler.
Streaming Analytics	Kinesis + Flink + Firehose + Athena	Ch18	Real-time trending, live dashboard, abandonment detection, data lake ingestion.
Observability	CloudWatch + X-Ray	Ch14	Metrics, logs, alarms, distributed tracing, anomaly detection across all services.
CI/CD	CodePipeline + CodeDeploy	Ch16	Every commit: build -> test -> blue/green deploy -> auto-rollback on error.
IaC	CDK (TypeScript)	Ch15	100% of infrastructure defined as code. Self-mutating pipeline deploys updates.
Cost Control	Savings Plans + Graviton + Tagging	Ch19	\$47K/month -> \$22K/month (-52%). Per-service cost visibility. FinOps dashboard.
High Availability	Aurora Global DB + Route 53 Failover	Ch20	99.99% uptime target. < 5 min RTO. < 1 second RPO. Monthly chaos engineering.

20-20

Chaos Engineering Lab — Creating a FIS Experiment

In this hands-on lab you will create an AWS FIS experiment that terminates 50% of product-svc ECS tasks in us-east-1a, then verify that ShopFlow continues to serve traffic from tasks in the other two AZs.

ECS Task Termination Experiment

■ Experiment Template Configuration

Description

Terminate 50% of product-svc tasks in us-east-1a to validate AZ resilience

■ Actions

Action 1 name

terminate-product-svc-tasks

Action type

aws:ecs:stop-task ■

Target: ECS tasks

Select by tag: Service=product-svc, AZ=us-east-1a. Selection mode: PERCENT, percentage: 50

Start delay

0 seconds

Duration

5 minutes

■ Stop Conditions

Stop condition

CloudWatch alarm: shopflow-api-5xx-rate-alarm (fires if 5xx > 1%). If alarm fires, FIS stops the experiment immediately.

■ IAM Role

Execution role

arn:aws:iam::ACCOUNT:role/FIS-ShopFlow-ExecutionRole (needs ecs:StopTask, cloudwatch:DescribeAlarms permissions)

■ ALWAYS test FIS experiments in staging before running in production. Set a CloudWatch stop condition on every experiment — this is your safety net if the chaos causes more damage than expected.

Create template

warn Chaos Engineering Safety Rules

ShopFlow follows three rules for every chaos experiment: (1) Always have a stop condition linked to a CloudWatch alarm — if the experiment makes things worse, FIS stops automatically. (2) Start in staging. Run in production only after staging passes. (3) Run during business hours with the SRE team monitoring — never schedule unattended chaos experiments for overnight or weekends.

20-21

Resilient Async Processing — SQS DLQ and Retry Strategy

ShopFlow's order processing pipeline uses SQS between checkout-svc and order-svc. Resilience in async processing requires careful configuration of visibility timeout, retry count, and dead-letter queue.

SQS Setting	ShopFlow Value	Why
Visibility timeout	300 seconds (5 min)	order-svc Lambda max timeout = 60s. Visibility timeout must be > 6x Lambda timeout
Message retention	14 days	If order-svc is down for days (extreme scenario), messages wait rather than being lost

SQS Setting	ShopFlow Value	Why
Receive message wait time (long polling)	20 seconds	Long polling reduces empty receive calls by 95%, lowering SQS costs and reducing latency.
Dead-letter queue (DLQ) max receive count	5	After 5 failed processing attempts, move to DLQ. order-svc team gets PagerDuty alert.
DLQ retention	14 days	Engineers have 2 weeks to investigate DLQ messages, fix the bug, and reprocess.
Lambda concurrency limit	100 (order-svc)	Prevents Lambda from scaling to 1,000 concurrent executions and overloading the database.

20-22

Multi-Region Architecture — ShopFlow eu-west-1 Warm Standby

This section describes exactly what runs in eu-west-1 at steady state, what gets activated during a failover, and how long each activation step takes.

Component	us-east-1 (Primary)	eu-west-1 (Warm Standby)	Failover Activation
Route 53	Primary failover record: api.shopflow.com	Secondary failover record: api.shopflow.com	Automatic. Route 53 detects a health check failure in 30s.
ALB	Fully active. Handles 100% of production traffic.	Zero targets registered. ALB reconfigures itself as targets when they start up.	Automatic. ALB reconfigures itself as targets when they start up.
ECS Fargate	Full scale: 6-30 tasks per service.	Minimum scale: 1 task per service (cost optimization).	Automatic. ECS scales up to full scale in ~5 minutes.
Aurora PostgreSQL	Writer + 2 readers. Handles all writes.	Global secondary: 1 reader (replica from us-east-1).	Manual. Promote secondary to primary in ~60s.
ElastiCache Redis	3-node cluster (full).	1-node cluster (minimum for availability).	Automatic. Global Tables serve as session backup. Redis scales up in ~5 minutes.
DynamoDB	Primary region. All writes go here.	Global Table replica. Receives all writes.	Automatic. DynamoDB Global Tables are always active.
Kinesis	Full streams: shopflow-user-events, shopflow-order-events.	Not replicated (streaming is idempotent).	Post-failover deployment. Kinesis streams in eu-west-1 within 15 minutes.

20-23

RDS Proxy — Protecting Aurora During Traffic Spikes

Aurora PostgreSQL has a maximum connection limit (~5,000 for r6g.2xlarge). ShopFlow's 30 ECS tasks each maintaining a 10-connection pool = 300 connections normally. During Black Friday scale-out to 150 tasks: 1,500 connections. That's fine. But Lambda functions connecting directly to Aurora on every cold start is a problem.

Scenario	Without RDS Proxy	With RDS Proxy
ECS tasks (steady state, 30 tasks)	300 connections to Aurora — fine	300 connections to RDS Proxy — proxy uses 50 connections
ECS tasks (Black Friday, 150 tasks)	1,500 connections to Aurora — approaching limit	1,500 connections to RDS Proxy — proxy still uses ~200 connections
Lambda functions (checkout, or other processors)	Each invocation opens a new connection to Aurora.	Lambda connects to RDS Proxy (pre-warmed connection). RDS Proxy manages connections to Aurora.
Aurora writer failover	All 1,500 connections must reconnect simultaneously.	RDS Proxy detects connection storm, reconnects transparently. Application unaffected.

20-24

The Cost of High Availability — What You Pay for Resilience

Every HA feature has a cost. ShopFlow pays \$3,820/month for its resilience infrastructure above and beyond what a single-AZ, single-region deployment would cost. Here is the breakdown.

HA Feature	Monthly Cost	Protection	Worth It?
Aurora Multi-AZ (2 reader replicas)	\$940/month (2 x r6g.2xlarge)	AZ failure. Reader failover < 30s. Read scalability.	YES — Aurora is a reader-heavy service.
ElastiCache 3-node vs 1-node	\$350/month (2 additional nodes)	AZ failure. Redis getoutofsync failover to replica < 60s.	YES — Single Redis is a single point of failure.
NAT Gateways x3 (one per AZ)	\$210/month (3 AZs x \$70)	AZ failure. Traffic in other AZs use their own AZ NAT Gateway.	YES (hot cross-AZ) NAT Gateway (hot cross-AZ) NAT Gateway.
Aurora Global Database (eu-west-1)	\$380/month (application on eu-west-1 reader)	Region failure. 5 min RTO.	YES — Black Friday: 1 minute of downtime is unacceptable.
eu-west-1 warm standby (ECS, ALB, NAT)	\$120/month (minimum 1 ECS task)	AZ failure. ALB + NAT Gateway traffic within 15 minutes.	YES — but only for Black Friday.

Q	A
What is the difference between Multi-AZ and Multi-Region?	Multi-AZ Multi-Region (ECS tasks, Aurora replicas, ElastiCache nodes) run in multiple physically separate Availability Zones within AWS (scale, notify, invoke Lambda). Route 53 health checks for CloudWatch alarm and CloudWatch actions within AWS (scale, notify, invoke Lambda). Route 53 health checks for CloudWatch alarm and CloudWatch actions within AWS (scale, notify, invoke Lambda).
Why use Route 53 health checks for CloudWatch alarm and CloudWatch actions within AWS (scale, notify, invoke Lambda)?	Route 53 health checks for CloudWatch alarm and CloudWatch actions within AWS (scale, notify, invoke Lambda). Route 53 health checks for CloudWatch alarm and CloudWatch actions within AWS (scale, notify, invoke Lambda).
What is connection pooling and why does ShopFlow need Redis?	Connection pooling pre-opens database connections and reuses them across requests, instead of opening a new connection for every request. ShopFlow uses Redis for connection pooling.
How do you handle data consistency during a global database failover?	Aurora Global Database uses two-page-level replication with < 1 second lag. During failover, there is a brief period of data inconsistency.

High availability requires that every layer in the stack has its own health check. If ALB does not know a task is unhealthy, it keeps sending traffic to it. If Route 53 does not know the region is down, it keeps sending users to it.

Layer	Health Check	Healthy Criterion	Unhealthy Action
ECS task level	Docker HEALTHCHECK in Dockerfile	Return 200 with http/5/localhost:8080/actuator/health	ECS task is UNHEALTHY. Restarts the container.
ALB target group	HTTP GET /actuator/health on port 8080 every 30 seconds	Return 200	Target deregistered from ALB. No more traffic.
Route 53 (region)	HTTPS GET api.shopflow.com/actuator/health every 30 seconds	Return 200 within 30 seconds	Multi-Region AWS Solution fails (30 seconds): primary region becomes secondary.
Aurora (writer)	Internal RDS health monitoring (not user-configured)	Writes accepted	Aurora writes with 30 second delay to a reader to write to primary.
ElastiCache (Redis)	Internal ElastiCache monitoring + application-level Redis ping	Application responds to PING in 1 second	ElastiCache promotes replica to primary. DNS updates.

```
// Spring Boot: /actuator/health endpoint – provide rich health status // build.gradle:
implementation "org.springframework.boot:spring-boot-starter-actuator" // application.yml –
expose health endpoint with details management: endpoint: health: show-details: when-authorized
# show details only to authenticated requests probes: enabled: true # enables
/actuator/health/liveness and /actuator/health/readiness health: db: enabled: true # Aurora:
SELECT 1 check redis: enabled: true # Redis: PING check diskspace: enabled: false # Fargate has
no persistent disk; disable // Result: GET /actuator/health // { "status": "UP", //
"components": { // "db": { "status": "UP", "details": { "database": "PostgreSQL" } }, //
"redis": { "status": "UP", "details": { "version": "7.0.15" } } // } // } // If Aurora is DOWN:
status = "DOWN", ALB health check fails, task deregistered
```

Sometimes you want to gradually shift traffic to a new region (testing latency, validating DR readiness) without a hard failover. Route 53 weighted routing enables this.

Migration Step	us-east-1 Weight	eu-west-1 Weight	Duration	What to Monitor
Baseline (normal operation)	100	0 (standby not receiving traffic)	Ongoing	ALB metrics us-east-1: error rate, p99 latency
Smoke test (initial validation)	99	1	15 minutes	Check eu-west-1 error rate. Any 5xx from cold starts?
Canary (5% to EU)	95	5	30 minutes	Compare p99 latency: us-east-1 vs eu-west-1. EU should be lower.
Gradual shift	50	50	2 hours	Monitor both regions equally. Auto-scaling behavior in eu-west-1.
Full DR validation	0	100	30 minutes	Run full load test: all traffic on eu-west-1. Verify Aurora Global Database.
Failback (normal)	100	0 (back to standby)	Immediate	Verify us-east-1 resumes cleanly. Check Kinesis consumer lag.

ShopFlow automates first-line recovery actions so the on-call SRE wakes up to a system that has already attempted self-healing. EventBridge rules trigger Lambda functions when CloudWatch alarms fire.

Alarm	Trigger Condition	Auto-Recovery Lambda Action	SRE Notified?
ECS task below desired	ECS RunningTaskCount < DesiredCount	Can do a forceful update	Yes
Aurora connection pool	RDS DatabaseConnections > 4,000	Can do a manual rds-proxy:ModifyDBProxyTargetDBParameterGroup	Yes
Kinesis iterator age > 60	Kinesis IteratorAgeMilliseconds max > 60,000	Can do a forceful lambda parallelization	Yes
ElastiCache > 90% mem	ElastiCache DatabaseMemoryUsagePercent > 90	Can do a forceful lambda parallelization	Yes

20-29

Hands-On Lab — Run a Well-Architected Review

AWS Well-Architected Tool is a free console tool that guides you through the 6 pillars with targeted questions and produces a prioritized list of improvements. ShopFlow runs this review before each Black Friday and after each major architecture change.

aws > AWS Well-Architected Tool > Create workload

ShopFlow Production Workload Review

Workload Definition

Workload name

ShopFlow Production

Industry type

Retail and ecommerce

Environment

Production

AWS Regions

us-east-1 (primary), eu-west-1 (DR)

Review Configuration

Operational Excellence pillar

Security pillar

Reliability pillar

Performance Efficiency pillar

Cost Optimization pillar

Sustainability pillar

Time to complete all 6 pillars: ~3 hours (first time). Annual re-review: ~1 hour (only questions with changed answers). Well-Architected Tool generates a prioritized improvement plan with "High Risk" and "Medium Risk" items. Address all High Risk items before Black Friday.

Create workload

20-30

Resilience Testing Checklist

You cannot trust resilience you have not tested. ShopFlow runs these tests quarterly in staging and annually in production (during off-peak hours with the SRE team on standby).

Test	How to Run	Expected Result	Failure Mode (if not tested)
ECS task termination (AZ-specific)	aws ecs stop-task on all tasks in new task set (annual other 2M AZs)	Stop task successfully	Discovery of production failure that causes task set to fail
Aurora failover test	aws rds failover-db-cluster --db-InstanceIdentifier shop80-staging-aurora	Read replica becomes writer	Read replica becomes writer
Redis failover	aws elasticache test-failover --replication-group-id shop10-redis-staging	Read replica becomes writer	Read replica becomes writer
Route 53 DNS failover	Manually set Route 53 health check to unhealthy (or add new health check)	Route 53 fails over to secondary	Route 53 fails over to secondary
Aurora Global promote	aws rds failover-global-cluster --instance-id shop10-redis-staging	Read replica becomes writer	Read replica becomes writer

20-31 You Are Now an AWS E-Commerce Architect

Over 20 chapters and hundreds of pages, you have built ShopFlow from a blank AWS account to a production-grade e-commerce platform capable of serving 1 million monthly active users, handling 10,000 requests per second on Black Friday, and recovering from a full AWS region failure in under 5 minutes.

Skill	What You Can Now Do
Architecture	Design multi-tier, multi-AZ AWS architectures for e-commerce. Choose the right service (Aurora vs DynamoDB).
Infrastructure as Code	Write CDK stacks (TypeScript or Python) that deploy complete, reproducible AWS environments. Use CDK Pipelines for automated deployments.
Observability	Build CloudWatch dashboards, set meaningful alarms, trace requests with X-Ray, and diagnose production incidents.
Security	Implement defense-in-depth: WAF, Cognito JWT auth, IAM least-privilege, VPC network isolation, KMS encryption.
Cost Optimization	Reduce cloud costs 40-60% without sacrificing performance: Savings Plans, Graviton, right-sizing, VPC Endpoints.
Reliability	Design for failure at every layer: circuit breakers, retry with backoff, health checks, auto-scaling, Multi-AZ, and more.
CI/CD	Build fully automated pipelines that go from code commit to production blue/green deployment with automatic rollback.

info Your Next Steps

To deepen your AWS expertise: (1) Take AWS Certified Solutions Architect Associate — the 20 chapters of this tutorial cover 80% of the exam content. (2) Deploy ShopFlow to your own AWS account using the CDK code from Chapter 15. (3) Join AWS Community Builders and re:Post — the community accelerates learning faster than any course. (4) Read the AWS Well-Architected Framework whitepaper — it is free and 60 pages of concentrated AWS best practices.

20-32 Capacity Planning — Black Friday at 10,000 req/s

High availability is not just about surviving failures — it is also about surviving success. ShopFlow's biggest availability risk on Black Friday is not an AWS outage: it is running out of capacity because the team underestimated peak load. Here is the 8-week pre-Black-Friday capacity planning process.

Week	Activity	Output
Week -8 (early October)	Baseline measurement: export last 3 Novembers CloudWatch metrics	Baseline. Calculate: Peak Black Friday Peak = 10,000 req/s
Week -6	Traffic modeling: apply growth factor (YoY growth rate x market share increase)	Staging environment. Defined task set, and derivatives
Week -5	CDK capacity configuration: update ECS max task counts, Kinesis ProvisionedThroughput, ElastiCache CacheNodes	CDK Stack. ProvisionedThroughput, ElastiCache CacheNodes
Week -4	Load testing in staging: run load test at base scenario (10,000 req/s) for 2 hours	Load test results. Identified and fixed a corner case
Week -2	Production capacity pre-scale: deploy Black Friday CDK configuration	Production ready. SRE (high availability) published. Metrics dashboard
Week -1 to BF	Daily capacity review: check auto-scaling has room to grow.	Black Friday readiness. SRE (high availability) published. Metrics dashboard

20-33 Ops Runbook — Responding to an AZ Failure

AZ failures are rare but they happen. This runbook covers the first 15 minutes of an AZ failure response. The good news: ShopFlow's architecture handles most of this automatically. Your job is to verify and stay out of the way.

■ AZ Failure Response (us-east-1a degraded)

1. ('Minutes 0-2 — PagerDuty fires (automated)', '# What AWS is doing automatically:\n# - ALB detects unhealthy targets in us-east-1a (health checks failing)\n# - ALB stops routing new requests to tasks in us-east-1a (within 30-60s)\n# - ECS service scheduler detects running < desired count\n# - ECS launches replacement tasks in us-east-1b and us-east-1c\n\n# Your first action: DO NOT PANIC. Check the dashboard.\n#\n<https://console.aws.amazon.com/cloudwatch/home#dashboards:name=ShopFlow-Overview>)

2. ('Minutes 2-5 — Verify ALB is routing around failed AZ', '# Check ALB target health per AZ\naws elbv2 describe-target-health \\\n --target-group-arn ARN \\\n --query "TargetHealthDescriptions[*].{AZ:Target.AvailabilityZone,Health:TargetHealth.State}"\n\n# Expected: us-east-1a targets show "unhealthy", us-east-1b/1c show "healthy"\n# GOOD: ALB has already stopped sending traffic to us-east-1a')

3. ('Minutes 5-10 — Verify ECS is scaling replacement tasks', '# Check ECS task counts are recovering\naws ecs describe-services \\\n --cluster shopflow-prod \\\n --services product-svc order-svc checkout-svc \\\n --query "services[*].{service:serviceName,running:runningCount,desired:desiredCount}"\n\n# Expected: running < desired (ECS is launching new tasks)\n\n# If still below after 5 min: check ECS events for placement failures\naws ecs describe-services \\\n --cluster shopflow-prod --services product-svc \\\n --query "services[0].events[:5]")

4. ('Minutes 10-15 — Check Aurora and ElastiCache', '# Aurora: did it failover to a reader in a healthy AZ?\naws rds describe-events \\\n --source-identifier shopflow-aurora-prod \\\n --duration 30\n\n# Look for: "Multi-AZ instance failover completed"\n\n# Check current Aurora writer AZ\naws rds describe-db-instances \\\n --query "DBInstances[?contains(DBInstanceIdentifier, `shopflow`)].{id:DBInstanceIdentifier,az:AvailabilityZone,role:MultiAZ}"\n\n# ElastiCache: check if primary failed over\naws elasticache describe-events --duration 30\n\n# If ElastiCache primary was in us-east-1a: failover to replica in 1b/1c is automatic < 60s')